QuickBite — National Rollout Design Brief

Context and planning figures

QuickBite is growing from 5,000 users and ~40 restaurants in one city to 2 million users nationally in 12 months: 400×, ~65% compound monthly growth. Engineering is one team of fewer than 15. These figures are **planning figures, not QuickBite facts.**

Input	Value
Daily active users	20% of users ≈ 400k
Orders	~100k/day; the 2-hour dinner peak carries ~30% (≈4 orders/s)
Restaurants	~16,000 (today's 1 per 125 users)
Peak browse reads	~4.8M in the peak window ≈ 670/s (~160 per order)
Riders	~35-minute order lifecycle → ~9,000 active at peak; 5 s pings ≈ 1,800 writes/s

Part 1 — Requirements Framing

Functional requirements

#	Persona	Requirement	Priority
FR1	Customer	Log in with phone number (OTP)	Must
FR2	Customer	Check serviceability by pincode; if unserved, join a demand poll	Must
FR3	Customer	Add, delete and manually enter addresses, including for someone else	Should
FR4	Customer	Browse nearby restaurants and menus; filter by price, rating, preference	Must
FR5	Customer	Add to cart, pay online or by COD, add instructions and place the order; cancel before pickup, with automatic refunds on rejection or cancellation	Must
FR6	Customer	Track the order live; notified on each status change	Must
FR7	Customer	Share a restaurant or dish; rate after delivery	Could
FR8	Restaurant	Accept/reject orders; see payment status and instructions; mark ready	Must
FR9	Delivery partner	Receive an assignment, view locations, accept, pick up, update status	Must
FR10	Delivery partner	Share live location, confirm delivery, record COD collection	Must

Acceptance criteria. FR2: Given an unserved pincode, the app shows "not available yet" and a one-tap poll recording pincode and phone number. **FR5:** Given two Pay taps or a timeout retry with one idempotency key, exactly one order and one charge exist, and both responses return that order.

Five design questions

Question	QuickBite answer
Scale and growth	400× in 12 months (inputs above). Peak concurrency, not storage, is the problem: capacity is modelled on orders/s, active riders and pings/s, and each city passes a load test before launch.
Read vs write	≈670 browse reads/s vs ≈4 order writes/s at peak (~160:1); rider location is the exception at ≈1,800 writes/s. Reads need cache and replicas; pings need an in-memory store.
Never lost vs can drift	**Never lost:** orders, payments, refunds, COD confirmations, assignments, status history, accounts. **Can drift:** ratings, ranking, ETA, poll counts, delivery receipts, intermediate rider positions.

Question	QuickBite answer
Latency	**Baseline (measured today):** home list ~2.5 s. **Targets at 2M users, customer-perceived p95, measured in the app:** home ≤ 2.5 s; search, filter, add-to-cart ≤ 1 s; order and payment confirmation ≤ 3 s with visible progress; rider marker ≤ 6 s old.
Cost	Infrastructure plus third-party API cost below ~5% of order value, tracked per order by city; maps, SMS and push fees scale with usage.

Three NFRs that conflict

Conflict	Resolution
Latency vs freshness: cached menus make 2.5 s possible but may show sold-out items	Short TTLs; active invalidation for sell-outs; price and stock re-validated at checkout
Availability vs cost: 99.99% (~4.3 min downtime a month) needs redundant load balancers, replicas and zones (about 2–3× cost); 99.9% allows ~43 min	99.9% overall; **99.99% for order placement and payment**, where failure loses transactions; provider outages use pending-and-reconcile (Part 5)
Tracking accuracy vs battery, data and API cost: 2 s pings mean 2.5× the writes of 5 s	Ping every 5 s; marker ≤ ~6 s old (~33 m at 20 km/h)

Part 2 — Data Model Driven by Access Patterns

Access patterns

#	Access pattern	Entity (index)	Served from	Consistency
1	Pincode serviceable?	Pincodes (pincode)	Cache	Bounded staleness
2	Restaurants near address	Restaurants (pincode, geo index)	Cache, else replica	Bounded staleness
3	Restaurant menu	MenuItems (restaurant_id)	Cache; photos on CDN	Bounded; sold-out invalidated actively
4	Cart price and stock	MenuItems (item_id)	Primary, bypassing cache	Strong
5	Place order	Orders, OrderItems, Payments, CheckoutRequests	Primary, one transaction	Strong, durable
6	My order's status	Orders (order_id)	Primary for 60 s, then replica; never cached	Read-your-writes
7	Advance order status	Orders, OrderStatusEvents	Primary, one transaction	Strong, append-only
8	Find riders; record location	Ephemeral location	In-memory geo store	Seconds-stale; dropped ping fine
9	Assign one rider	Assignments (order_id unique)	Primary, conditional write	Strong
10	Restaurant's incoming orders	Orders (restaurant_id, current_status, placed_at)	Primary, plus push	Strong
11	My past orders	Orders (user_id, placed_at DESC)	Replica, cursor pagination	Eventual

Core entities

Users have many Addresses and Orders. Restaurants have many MenuItems and Orders, plus avg_rating and rating_count updated asynchronously. An Order has many OrderItems (qty, price_at_order), many status events, one Payment and at most one Assignment (order_id unique, with a lease). CheckoutRequests hold a unique (user_id, idempotency_key) and the resulting order_id. OrderItems resolve the order ↔ menu-item many-to-many and freeze the price paid; orders point to an address, so ordering for someone else works. Each table has one owning module that alone writes it.

The trickier relationship: order status

A link such as user_id → order_id never changes; order status does. It is a **value with a timeline** (overwriting loses who changed it and when, which disputes and refunds need), it has **rules** (Delivered cannot return to Placed), four actors write it (restaurant, rider, system, support), and its timestamps drive analytics.

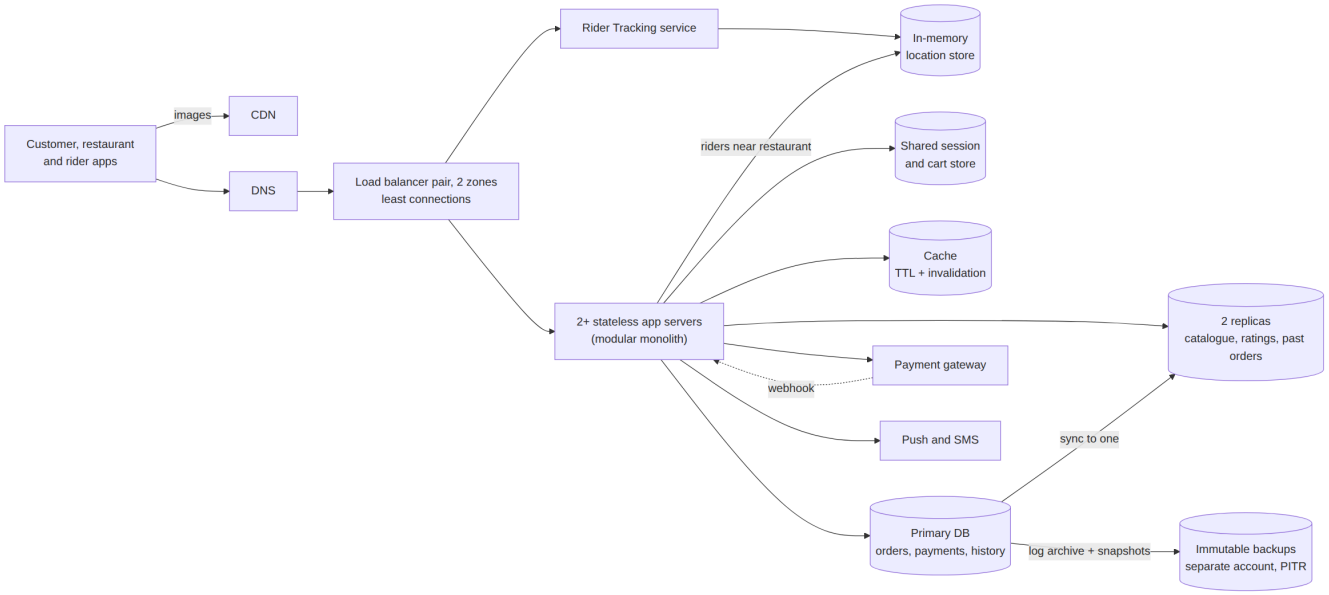
`Orders.current_status` and `version` give fast reads. `OrderStatusEvents` is an append-only ledger recording order, sequence, command, from/to status, actor, reason and timestamp. Each transition is one transaction: a compare-and-set update on Orders (only if the current status equals the expected one) plus the event insert. Unique `(order_id, sequence)` and `(order_id, command_id)` make a duplicate command (a double-tapped Accept, a retry) return the earlier result, and an illegal or racing transition is rejected.

Legal transitions: Placed → Accepted | Rejected; Accepted → Preparing; Preparing → Ready; Ready → PickedUp (needs an active assignment); PickedUp → Delivered. Any state before PickedUp can move to Cancelled. Rejected and Cancelled trigger refunds.

Part 3 — Architecture Decisions

Decision: modular monolith plus one separate service

For the national rollout, QuickBite uses a **modular monolith for the transactional core and one separately deployed service, Rider Tracking.**



- **Independent scaling:** only rider location has a different load shape: ~1,800 writes/s against ~4 order writes/s, tolerant of a dropped ping. It needs an in-memory store; orders need a durable relational database. Everything else scales together with dinner traffic.
- **Team size:** one team of fewer than 15. Splitting the core into marketplace, commerce and delivery services pays off only with a durable owning squad and on-call rotation each; otherwise it adds network hops and cross-service checkout transactions first.
- **Trade-off accepted:** one bad deploy can affect every core feature, and core scaling is coarse. Strict modules keep extraction cheap.
- **Next extraction:** Catalogue and Search, when campaign browsing and geo queries compete with checkout for CPU.

When and how to scale

Scaling is triggered when any signal holds **on three consecutive peak days**, so one unusual spike does not trigger it. Thresholds mark when customers approach a bottleneck, not merely when a server is busy.

Signal	Threshold	Why it matters
p95 home-screen latency (customer-perceived)	> 2.5 s	Breaks the NFR
Order success rate	< 99%	Revenue and trust leak at checkout
Error rate	> 1%	Customers see failures first
CPU or DB connections	> 70–75% sustained during peak	Headroom gone; a brief spike is not a reason
Replica lag	> 2–3 s and increasing	Stale reads are reaching users

Vertical first for the database primary: simple, no code change. **Horizontal for the app tier from day one:** at least two servers in two zones, workable because they are stateless (Part 5). Sharding is deferred: ~4 orders/s is far below one primary's capacity.

Part 4 — Request Flow and Traffic Management

Opening the app and finding nearby restaurants

1. The app requests restaurants for the saved pincode over HTTPS; **DNS** resolves the API domain to the load balancer, which skips unhealthy servers and picks the one with the fewest active connections.
2. A **stateless app server** validates the login token in the shared session store, then checks the cache.
3. **Hit:** milliseconds. **Miss:** a replica answers through the location index and fills the cache.
4. The app renders 20 restaurants, then lazy-loads photos from the CDN.

The 2.5 s target is customer-perceived p95; server p95 is a diagnostic. Planned split: ~0.6 s network, ~0.3 s server, ~1.6 s rendering and images (to validate with real-user monitoring), so lazy loading matters more than server tuning.

Live rider location: latency, bandwidth or both?

Mostly neither; the pressure is request volume.

- **Bandwidth:** ~200 bytes × ~1,800/s is under 3 Mbps in total, ~40 bytes/s per trip. Negligible.
- **Latency:** a 5 s interval plus ~1 s round trip keeps the marker ≤ ~6 s old (~33 m at 20 km/h). Acceptable.
- **Volume:** ~1,800 writes/s plus similar customer reads, each with map and push cost; this is why tracking is the one separate service.

Load balancing: least connections

Request durations vary widely: a menu read takes milliseconds, a payment call seconds, and tracking connections stay open. Round robin treats all requests as equal and would pile slow ones onto unlucky servers at dinner peak. **Least connections** routes each new request to the least busy server. Weights are unnecessary because the app servers are identical in size; they are added only if sizes diverge.

Part 5 — Statefulness and Reliability

State that must survive a server failure

State	Lives in	Failure it prevents
Login session, incl. OTP in progress	Shared session store	Logout at payment
Cart and order draft	Shared store; snapshotted to the database at checkout	Empty cart
Checkout and idempotency state	CheckoutRequests row on the primary, written before payment	A retry creating a second order
Payment-provider outcome	Payments row, set by webhook and a reconciliation job for attempts pending beyond ~2 min	Charged, restaurant never told
Courier assignment	Assignments row with a lease, updated by conditional writes	One order, two riders
Rider position	Shared in-memory store	Lost marker on reconnect

The test: **if this server vanished now, would a customer lose something or see a contradiction?** Login and checkout fail it unless their state is shared and durable; nearby search passes.

Why sticky sessions are not enough

Adding servers is like adding waiters who each keep orders on a private notepad: the next request may reach a waiter who never saw the customer. Sticky sessions (always the same waiter) hide this until that waiter vanishes through a crash, deploy or scale-in; then the customer is logged out at payment or charged with no order recorded. The fix is one shared notebook: session, cart and checkout state live in shared or durable stores, so any server can serve any request.

Checkout must be idempotent and recoverable across server failures. QuickBite persists the checkout attempt before calling the payment provider and ties it to a unique idempotency key, so a retry cannot create a second order or charge. If the payment outcome is temporarily unknown, the app shows "Confirming your order" instead of a failure, and the payment webhook or a reconciliation job completes the order later. This keeps payment and order state durable while the application layer stays stateless.

Part 6 — Performance and Consistency

Consistency boundaries

Strong and durable (primary): the never-lost list (Part 1) plus checkout price and stock. **Bounded staleness (cache or replica):** rankings and ratings (≤ 15 min), catalogue presentation (≤ 5 min), nearby lists (≤ 60 s), ETA and rider position (≤ 6 s), delivery receipts.

Indexing

Index	Serves	Without it
Restaurants (pincode) + geo index	Pattern 2	Each home load scans ~16,000 restaurants at ~670 reads/s
Orders (restaurant_id, current_status, placed_at)	Pattern 10	Scan and sort ~36M orders a year at dinner rush
Orders (user_id, placed_at DESC)	Pattern 11	Scanning other customers' orders
Deliberately none: Orders.instructions, Ratings.text	Free text is never searched	Every index slows writes on the most critical table

Caching by the cost of stale data

What	Strategy	Reasoning
Serviceability	Invalidated on zone change; 10 min safety TTL	A wrong answer sells into an unserved area
Nearby restaurant list	60 s TTL	Open state, ETA and ranking may lag; checkout re-validates
Menu content	5 min TTL	Price and stock re-validated at checkout
"Sold out" flag	Active invalidation: the restaurant's tap deletes the entry	A gone dish is ordered, then rejected and refunded
Promotion rules	Cached with a version recorded on the order; checkout re-prices	Stale eligibility changes price
Ratings, "popular" labels	15 min TTL	Stale cost near zero
Order and payment status	**Never cached**	A wrong answer causes duplicate orders or double charges

Every TTL carries ±10–20% jitter so entries do not expire together. A ≥ 95% hit ratio leaves ≤ ~35 reads/s for replicas at peak (planned).

Replicas and read-your-writes

An order placed 10 seconds ago must not be read from a replica: lag turns it into "order not found", which looks like failure. For **60 seconds** after placement the customer's own order reads from the primary (~50 reads/s at a 5 s refresh: trivial). After that, status reads use replicas, uncached, with the lag alert protecting the 6 s tolerance. Cancel, assign and refund decisions always read the primary.

What replication does not protect against

Replication covers **hardware and zone failure**, not **human and logical errors** (a deleted table, a bad migration): the mistake reaches every replica in milliseconds. That needs **point-in-time recovery (PITR)** backups in a separate account with immutable retention, and restores rehearsed quarterly against the RTO. Commits wait for one replica in another zone, making near-zero RPO credible for orders and payments.

Data	RPO	RTO	Mechanism
Orders, payments, refunds, COD, assignments, status transitions	Near-zero	≤ 1 min for machine or zone loss; ≤ 1 hr after corruption	Synchronous replica, automatic failover, continuous log archiving
Less critical data (ratings, poll votes, analytics)	≤ 5 min	≤ 1 hr	Frequent snapshots; aggregates rebuilt from source events
Rider location	None	~5 s	Rebuilt from the next pings

Part 7 — Incident Response Drill

a) The load balancer fails for 4 minutes at dinner rush

For four minutes customers could not reach the app, exposing roughly 1,000 order attempts (~4 orders/s) and using over 90% of the monthly downtime budget of a 99.99% order path (~4.3 minutes). One load balancer sat in front of every request, so healthy servers behind it were unreachable. Mitigation is to replace it, re-point DNS and show an in-app status message. The permanent fix is a redundant pair across two zones with automatic failover within 60 seconds, alerting within a minute and a quarterly failover drill, owned by Platform Engineering, with Product validating the recovery target.

b) The order is saved on the primary, but the app shows "order failed" from a replica

The data is safe, but the behaviour is a design bug: a read-after-write hit a lagging replica and the UI treated "not found" as "failed". Customers who see "failed" retry, creating duplicate orders, double charges and refund work. Immediately, tell support, finance and restaurants the order is safe, refund duplicates and read status from the primary. Permanently, checkout returns the committed order, the customer's own order reads from the primary for 60 seconds, "failed" appears only for an authoritative terminal result (otherwise "Confirming your order"), and idempotency keys make retries return the same order. Engineering implements; Product owns the UX states.

c) A delete query wipes the Orders table and replication copies the wipe

Every replica repeated the delete within milliseconds, so active orders cannot be fulfilled, payments cannot be matched and ordering stops until restore. The weakness is treating replicas as backups: there was no point-in-time recovery (PITR), no separate backup boundary and no control over destructive statements. Mitigation is to stop writes, restore PITR to a new instance just before the delete and reconcile against payment-provider records before cutover. The permanent fix is continuous log archiving plus snapshots in a separate account with immutable retention, least-privilege access, approval for destructive changes and quarterly restore drills. Product defines the requirement (near-zero RPO and ≤ 1 hour restore RTO for orders and payments; ≤ 5 minutes RPO and ≤ 1 hour RTO for less critical data) and Engineering implements and tests it.

d) A cricket-match banner sends 50,000 requests at one menu the instant its cache entry expires

Every request missed the cache and hit the database together, a stampede of ~10,000 reads/s within ~5 seconds, about 15× the normal peak of ~670/s, slowing every screen and raising replica lag. The weakness is a hot key with synchronised expiry, no request collapsing and no campaign forecast. Mitigation is to refresh that key, extend its TTL and serve the last-known menu. The permanent fix is single-flight rebuilds (one request rebuilds while others wait), staggered expiry, pre-warming promoted menus and stale-if-error, which is safe because checkout re-validates price and stock. Engineering builds it; Marketing shares the campaign calendar at least 48 hours ahead (planned) so a forecast and pre-warm exist before launch, a process Product owns.

Part 8 — Executive Summary

QuickBite must grow from 5,000 to 2 million users in 12 months (400×). Peak dinner traffic, not average load, sets the design.

Two extra database copies. They do two jobs. If the main database fails, one takes over in about a minute instead of QuickBite going dark at dinner. They also absorb browsing reads (about 160 per order placed), protecting the 2.5-second load target. Cost: roughly 3× primary database compute (illustratively, $500 becomes about $1,500 a month, excluding backups and network). Copies are not backups: a copy repeats a deleted table, so point-in-time backups are funded separately.

No microservices for the national rollout. A competitor's blog describes a design built for a far larger engineering organisation. Splitting pays off when parts must scale independently or several teams must ship separately. QuickBite has one team of under 15 and one clear candidate, live rider tracking, separated from day one. Everything else stays one well-organised application: faster to build, easier to debug, cheaper to run. Further splits, starting with search, follow measured signals.

Stale screens, never stale payments. A rating or menu photo that is minutes old harms nobody, so those screens are cached. Payment confirmation, order status, and price and stock at checkout come from the source of truth, because a wrong answer means a double charge or a vanished dish. Order placement and payment target 99.99% availability; everything else, 99.9%.

Decisions requested. Infrastructure and third-party API cost stays below ~5% of order value. Approval is requested for the two database copies, separately funded backups, and the 99.99% target for the order and payment path.